OptiCrop: Smart Agricultural Production Optimization Engine

Project Description:

OptiCrop harnesses Machine Learning to provide intelligent, data-driven agricultural recommendations, offering farmers, agronomists, and policymakers a fast and personalized crop optimization experience. The platform includes a Crop Recommendation module, which predicts the most suitable crop from 22 options based on seven soil and climate parameters (Nitrogen, Phosphorous, Potassium, Temperature, Humidity, pH, and Rainfall) using an ensemble of Random Forest, Decision Tree, and SVC classifiers; a Crop Suitability Assessment module, which evaluates whether a chosen crop is compatible with current environmental conditions using a weighted seven-metric scoring system with remediation steps; a Fertilizer Recommendation module, which identifies soil NPK deficiencies and recommends fertilizer type and dosage; a Plant Disease Detection module, which classifies crop diseases from observed symptoms or an uploaded leaf image; and a Research Analytics Dashboard, which visualises crop-environment patterns using Chart.js for agricultural researchers and policy makers.

Built using Python's built-in http.server (ThreadingHTTPServer) with no external web framework, OptiCrop processes user inputs via REST API endpoints to deliver fast, high-quality agricultural recommendations. Real-time weather data is fetched automatically using the Open-Meteo API (no API key required), which provides live temperature, humidity, and 30-day rainfall for any city or GPS location. The application runs locally at http://127.0.0.1:8000 and is accessible across the local network, requiring only Python 3.10+ and pip to set up.

Scenarios

Scenario 1 — Smart Crop Recommendation: A farmer enters soil and environmental details — Nitrogen (N=90), Phosphorous (P=42), Potassium (K=43), Temperature (20°C), Humidity (82%), pH (6.5), and Rainfall (202 mm) — on the Recommend page. The system processes these inputs through the trained ML ensemble (crop_model.joblib) and recommends the optimal crop with a confidence score, top candidate crops list, and fertilizer correction advice — all within under one second.

Scenario 2 — Crop Suitability Assessment: An agricultural extension officer selects a specific crop (e.g., Mango) and enters current environmental parameters. OptiCrop evaluates each of the seven soil and climate metrics against dataset-derived ideal ranges, computes a weighted overall suitability score, and returns a compatibility label (Excellent / Strong / Moderate / Poor) along with per-metric status and specific remediation steps to correct any deficiencies before planting.

Scenario 3 — Agricultural Research & Policy Planning: An agricultural researcher opens the Research Insights dashboard (insights.html). The dashboard fetches aggregated crop-environment statistics from /api/analytics and renders them using Chart.js — average NPK by crop (bar chart), crop distribution (donut chart), climate scatter plot (temperature vs. humidity), and rainfall by region. The researcher filters by crop, region, and season, then exports the filtered dataset as a CSV file for policy reporting.

Technical Architecture:

OptiCrop uses a modular three-tier architecture to deliver rapid, ML-driven agricultural recommendations. The frontend provides a responsive, dark-themed user interface built with HTML5, CSS3, and Vanilla JavaScript across five pages (index.html, recommend.html, disease.html, insights.html, contact.html), while the Python ThreadingHTTPServer backend orchestrates input validation, ML inference, and API calls. The backend interfaces with three trained Scikit-learn model bundles (crop_model.joblib, disease_model.joblib, image_disease_model.joblib), a Pandas-loaded CSV dataset (Crop_recommendation.csv — 2200 rows, 22 crops), and the Open-Meteo weather API for real-time climate data.

Note: OptiCrop does not use a relational database. All crop and disease data is stored in flat files (CSV and JSON). No ER Diagram is required. If a database is integrated in a future version, an ER Diagram will be added at that stage.

Pre-requisites:

- Python 3.10+ and pip: Python Documentation — https://docs.python.org/3/
- Scikit-learn ML Library: https://scikit-learn.org/stable/
- Pandas and NumPy for data processing: https://pandas.pydata.org/
- Joblib for model serialisation: https://joblib.readthedocs.io/
- Pillow (PIL) for image handling: https://pillow.readthedocs.io/
- TensorFlow-CPU (optional, for CNN image disease model): https://www.tensorflow.org/
- Open-Meteo API (no key required): https://open-meteo.com/
- HTML, CSS, JavaScript and Chart.js 4.4.1: https://www.chartjs.org/
- Git and GitHub for version control: https://docs.github.com/
- Kaggle account for dataset download: https://www.kaggle.com/

Project Workflow:

Milestone 1: Dataset Preparation and Model Selection

- Activity 1.1: Download the Crop_recommendation.csv dataset from Kaggle and perform Exploratory Data Analysis (EDA).
- Activity 1.2: Research and select the best classification algorithm from seven candidates for crop recommendation.
- Activity 1.3: Define the architecture of the application — interactions between frontend pages, server.py API routes, and ML models.
- Activity 1.4: Set up the development environment — install Python dependencies and verify the server runs correctly.

Milestone 2: Core ML Model Development

- Activity 2.1: Train and save the crop recommendation ensemble model (crop_model.joblib) and the fertilizer recommendation module.
- Activity 2.2: Train and save the plant disease classifier (disease_model.joblib) and the image-based disease model (image_disease_model.joblib).

Milestone 3: server.py Backend Development

- Activity 3.1: Implement all seven REST API routes in server.py — /api/recommend, /api/suitability, /api/fertilizer, /api/disease, /api/image-disease, /api/weather, /api/analytics — with input validation and graceful error handling.

Milestone 4: Frontend Development

- Activity 4.1: Design and develop the five HTML pages with a shared dark-themed CSS stylesheet, mobile-responsive layout, and GPS weather auto-fill.
- Activity 4.2: Implement dynamic JavaScript result rendering for recommendations, disease predictions, suitability scores, and analytics charts.

Milestone 5: Local Deployment and Testing

- Activity 5.1: Configure the local development environment and verify all dependencies are installed via requirements.txt.
- Activity 5.2: Run the application locally (python3 server.py) and test all seven API endpoints and five web pages end-to-end.

Milestone 6: Conclusion

Milestone 1: Dataset Preparation and Model Selection

In this milestone, we focus on preparing the crop recommendation dataset, performing exploratory data analysis, and selecting the optimal machine learning algorithm. This involves evaluating multiple classifiers and choosing the one that maximises prediction accuracy while maintaining fast inference speed for real-time farmer use.

Activity 1.1: Download the Dataset and Perform EDA

The Crop Recommendation Dataset is sourced from Kaggle and contains 2,200 rows with 8 columns: N, P, K, temperature, humidity, ph, rainfall, and label (crop name). The dataset covers 22 crops with exactly 100 samples each — perfectly balanced.

- Step 1 — Download from Kaggle: https://www.kaggle.com/datasets/atharvaingle/crop-recommendation-dataset and save as data/Crop_recommendation.csv.
- Step 2 — Load and inspect the dataset:

```
import pandas as pd
df = pd.read_csv('data/Crop_recommendation.csv')
print(df.shape) # (2200, 8)
print(df.dtypes) # 7 float64, 1 object (label)
print(df.isnull().sum()) # No missing values
print(df['label'].value_counts()) # 22 crops, 100 each
```

- Step 3 — Plot feature distributions and a correlation heatmap using Seaborn to understand value ranges per crop.
- Step 4 — Confirm class balance: all 22 crops have exactly 100 samples — no oversampling needed.
- Step 5 — Split into training (80%) and testing (20%) sets with stratified sampling:

```
from sklearn.model_selection import train_test_split
X = df.drop('label', axis=1)
y = df['label']
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42, stratify=y)
# Train: (1760, 7) | Test: (440, 7)
```

Activity 1.2: Select the Best Classification Algorithm

Seven classification algorithms were evaluated on the same train/test split. The goal was to find the algorithm with the highest accuracy while remaining fast enough for real-time inference (target: under 1 second per prediction).

- Train and evaluate: Decision Tree, Naive Bayes, SVM, Logistic Regression, KNN, XGBoost, and Random Forest.
- Compare accuracy scores on the held-out test set:

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
import joblib

# Results on test set (440 samples):
# Decision Tree : 90.00%
# Naive Bayes : 99.09%
# SVM : 10.68%
# Logistic Regression: 95.23%
# KNN : 97.50%
# XGBoost : 99.09%
# Random Forest : 99.55% <-- Selected (highest accuracy)
```

- Select Random Forest (n_estimators=350) as the primary model — highest accuracy (99.55%), robust to outliers, fast inference (~0.35s).
- Build an ensemble of RF + Decision Tree + SVC with probability averaging for final prediction, saved as crop_model.joblib using joblib.

Activity 1.3: Define the Application Architecture

- Draft the architectural diagram showing five layers: Presentation (5 HTML pages), API Gateway (server.py ThreadingHTTPServer), ML Inference (3 joblib model bundles), Data Store (CSV + JSON flat files), and External API (Open-Meteo weather).
- Detail frontend functionality: users enter soil/climate parameters, use GPS weather auto-fill, upload leaf images, and view results — all without page reload via Fetch API.
- Outline backend responsibilities: server.py handles GET (static files via safe_resolve) and POST requests (7 API routes), loads all models at startup, and returns structured JSON responses.
- Describe ML integration: crop_model.joblib loaded once at startup; predict() called per request with 9 features; result includes predicted crop, confidence %, and top 10 candidates.

Activity 1.4: Set Up the Development Environment

- Install Python 3.10+ and verify with python3 --version.
- Create and activate a virtual environment:

```
python3 -m venv opticrop_env
source opticrop_env/bin/activate # Linux / macOS
# OR
opticrop_env\Scripts\activate # Windows
pip install -r requirements.txt
```

- Install core ML dependencies:

```
pip install scikit-learn>=1.4.0 pandas>=2.2.0 numpy joblib>=1.4.0
pip install matplotlib>=3.8.0 seaborn>=0.13.0 Pillow>=10.0.0
pip install tensorflow-cpu>=2.14.0 # optional — only for CNN image model
```

- No API key setup required — OptiCrop uses Open-Meteo (free, open, keyless).
- Verify setup by running: python3 server.py — server should start at http://127.0.0.1:8000.

Milestone 2: Core ML Model Development

Milestone 2 focuses on training and saving all three ML model bundles that power OptiCrop's core features: crop recommendation, disease classification from symptoms, and disease classification from a leaf image.

Activity 2.1: Train the Crop Recommendation and Fertilizer Modules

- Run ml/train_model.py to train the RF + DT + SVC ensemble on the 9-feature crop dataset:

```
python3 ml/train_model.py
# Output: models/crop_model.joblib
# models/confusion_matrix.png
# Accuracy on test set: Random Forest 99.55%
```

- The joblib bundle contains: RandomForestClassifier (n=350), DecisionTreeClassifier, SVC (via Pipeline), LabelEncoder, StandardScaler, and feature list.
- Fertilizer recommendation is implemented as a rule-based module (get_fertilizer_recommendation()) in server.py — no separate training needed. It uses NPK gap analysis against crop-specific nutrient targets stored in a Python dict.

Activity 2.2: Train the Plant Disease Models

- Run ml/train_disease_model.py to train a Random Forest disease classifier on augmented disease profiles from opticrop-data.json:

```
python3 ml/train_disease_model.py
# Output: models/disease_model.joblib
# models/disease_confusion_matrix.png
# Features: crop (encoded), temperature, humidity, ph, symptoms (multi-hot)
```

- Data augmentation: each disease record in opticrop-data.json is augmented 10x with small Gaussian noise on numeric features to expand the training set.
- For image-based disease detection, run ml/train_image_model.py for the lightweight pixel-feature Random Forest (no GPU needed):

```
python3 ml/train_image_model.py
# Output: models/image_disease_model.joblib
# Uses pixel statistics (mean, std, percentiles per RGB channel) as features
```

Milestone 3: server.py Backend Development

Milestone 3 focuses on implementing the core logic of server.py, which serves as the backbone of OptiCrop. All seven API routes, static file serving, model loading, input validation, and error handling are implemented in this single self-contained file.

Activity 3.1: Implement All API Routes in server.py

Server Startup and Model Loading:

- All three model bundles are loaded at server startup — not per request — ensuring fast inference times:

```
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
import joblib, json, os

# Load models once at startup
crop_model = joblib.load('models/crop_model.joblib')
disease_model = joblib.load('models/disease_model.joblib')
image_model = joblib.load('models/image_disease_model.joblib')

server = ThreadingHTTPServer(('0.0.0.0', 8000), OptiCropHandler)
print('Server running at http://127.0.0.1:8000')
server.serve_forever()
```

Core API Routes:

- /api/recommend (POST) — accepts nitrogen, phosphorous, potassium, temperature, humidity, ph, rainfall, location, season as JSON; runs RF+DT+SVC ensemble; returns predicted_crop, confidence, candidates list, fertilizer object:

```
@route POST /api/recommend
data = json.loads(body)
features = [data['nitrogen'], data['phosphorous'], data['potassium'],
data['temperature'], data['humidity'], data['ph'],
data['rainfall'], data['location'], data['season']]
prediction = crop_model['rf'].predict([features])[0]
confidence = max(crop_model['rf'].predict_proba([features])[0]) * 100
return {'predicted_crop': prediction, 'confidence': confidence, ...}
```

- /api/suitability (POST) — evaluates 7 environmental metrics against dataset-derived ranges; returns suitability label, overall_score, per-metric breakdown, remediation steps.
- /api/fertilizer (POST) — NPK gap analysis; returns fertilizer type (Urea/DAP/MOP/None), nutrient_balance dict, soil_note.
- /api/disease (POST) — crop + symptoms + environment → RF classifier → predicted_disease, confidence, risk_level.
- /api/image-disease (POST) — raw image bytes + crop name → pixel-feature extraction → image_model.predict() → disease class.
- /api/weather (GET) — city name or lat/lon → Open-Meteo geocoding + forecast → temperature, humidity, 30-day rainfall.
- /api/analytics (GET) — loads Crop_recommendation.csv → aggregates by_crop, by_region, by_season stats → Chart.js data.

Input Validation and Error Handling:

- All POST endpoints wrap inference in try/except — return HTTP 400 for missing/invalid fields, HTTP 503 if model file is missing.

- safe_resolve() prevents directory traversal for all static file GET requests — only .html, .css, .js, .png, .jpg, .json, .ico, .svg, .webp are served.
- Graceful fallback in /api/recommend: if crop_model.joblib is missing, returns hardcoded common crops with fallback=True flag — UI never crashes.

Milestone 4: Frontend Development

Milestone 4 focuses on developing a professional, agriculture-themed dark-mode interface for OptiCrop across five HTML pages. All pages share a single styles.css stylesheet and use Vanilla JavaScript for dynamic result rendering via the Fetch API — no page reloads.

Activity 4.1: Design and Develop the User Interface

- Build five semantic HTML pages: index.html (Home), recommend.html (Scenarios 1 & 2), disease.html (Disease Check), insights.html (Research Dashboard), contact.html.
- Implement a shared dark-green agricultural theme in styles.css — dark backgrounds (#0d1f0f, #1a2e1a), green accent colours (#4ade80, #22c55e), clean card layouts with subtle borders.
- Add a mobile-responsive hamburger navigation menu visible on screens under 768px width.
- Use CSS Grid and Flexbox for the two-panel layout on recommend.html (input form left, results right) and for the four-chart analytics grid on insights.html.
- Implement GPS weather auto-fill: clicking 'Use my location' triggers browser geolocation API → coordinates sent to /api/weather → temperature, humidity, rainfall fields pre-filled automatically.

Activity 4.2: Dynamic Content Rendering with JavaScript

- Crop recommendation form: on submit, read all 9 fields, POST to /api/recommend, render predicted crop name, confidence bar, top candidates list, and fertilizer advice card — all without page reload:

```
document.getElementById('recommend-form').addEventListener('submit', async (e) =>
{
e.preventDefault();
const payload = { nitrogen: N.value, phosphorous: P.value, ...
temperature: temp.value, humidity: hum.value, ... };
const res = await fetch('/api/recommend', {
method: 'POST',
headers: {'Content-Type': 'application/json'},
body: JSON.stringify(payload) });
const data = await res.json();
renderRecommendation(data); // inject crop name, confidence, fertilizer
});
```

- Disease page: symptom tag selection updates a live array; on submit, POST to /api/disease; render disease name, confidence, risk level. Image upload panel POSTs raw file bytes to /api/image-disease.
- Analytics dashboard (insights.html): on page load, fetch /api/analytics → pass data to Chart.js to render bar chart (avg NPK by crop), donut chart (crop distribution), scatter plot (temperature vs. humidity), and bar chart (rainfall by region). Filter dropdowns re-fetch with query params and update all four charts simultaneously.
- CSV export button: downloads the currently filtered analytics data as a .csv file using Blob and URL.createObjectURL().

Milestone 5: Local Deployment and Testing

Milestone 5 focuses on deploying OptiCrop locally and verifying that all features work end-to-end. OptiCrop runs entirely on the local machine using Python's built-in ThreadingHTTPServer — no cloud hosting, no external web framework, and no API key is required to run the application.

Activity 5.1: Prepare the Local Environment

- Clone the repository and navigate to the project folder:

```
git clone https://github.com/RamanHarshitha/Smart_Bridge
cd Smart_Bridge-master
```

- Install all dependencies from requirements.txt:

```
pip install -r requirements.txt
# Core: scikit-learn, pandas, numpy, joblib, Pillow, matplotlib, seaborn
# Optional: tensorflow-cpu (only needed for CNN image disease model)
```

- Download Crop_recommendation.csv from Kaggle and place it in the data/ folder.
- Train the crop recommendation model:

```
python3 ml/train_model.py
# Generates: models/crop_model.joblib + models/confusion_matrix.png
```

- Train the disease classifier:

```
python3 ml/train_disease_model.py
# Generates: models/disease_model.joblib + models/disease_confusion_matrix.png
```

Activity 5.2: Run and Test the Application

- Start the local server:

```
python3 server.py
# Output:
# OptiCrop server running at http://127.0.0.1:8000
# Also accessible on LAN at http://192.168.x.x:8000
```

- Open a browser and navigate to http://127.0.0.1:8000 — the OptiCrop Home page loads.
- Test Crop Recommendation: enter N=90, P=42, K=43, temp=20, humidity=82, pH=6.5, rainfall=202 → verify rice is recommended with >90% confidence.
- Test Weather Auto-fill: enter 'Nellore' in the city field → verify temperature, humidity, rainfall are auto-populated from Open-Meteo API.
- Test Crop Suitability: select Mango with temperature=5°C → verify Poor suitability with remediation steps.
- Test Disease Detection: select Rice + brown lesions + yellowing symptoms → verify disease prediction with risk level.
- Test Image Disease: upload a leaf photograph → verify image-based classification returns a disease name.
- Test Analytics Dashboard: open insights.html → verify all four Chart.js charts load with 2200 dataset records.
- Test CSV Export: apply a crop filter → click Export Filtered CSV → verify correct filtered data downloads.

Exploring the Website's Web Pages:

Home Page (index.html) — http://127.0.0.1:8000/index.html

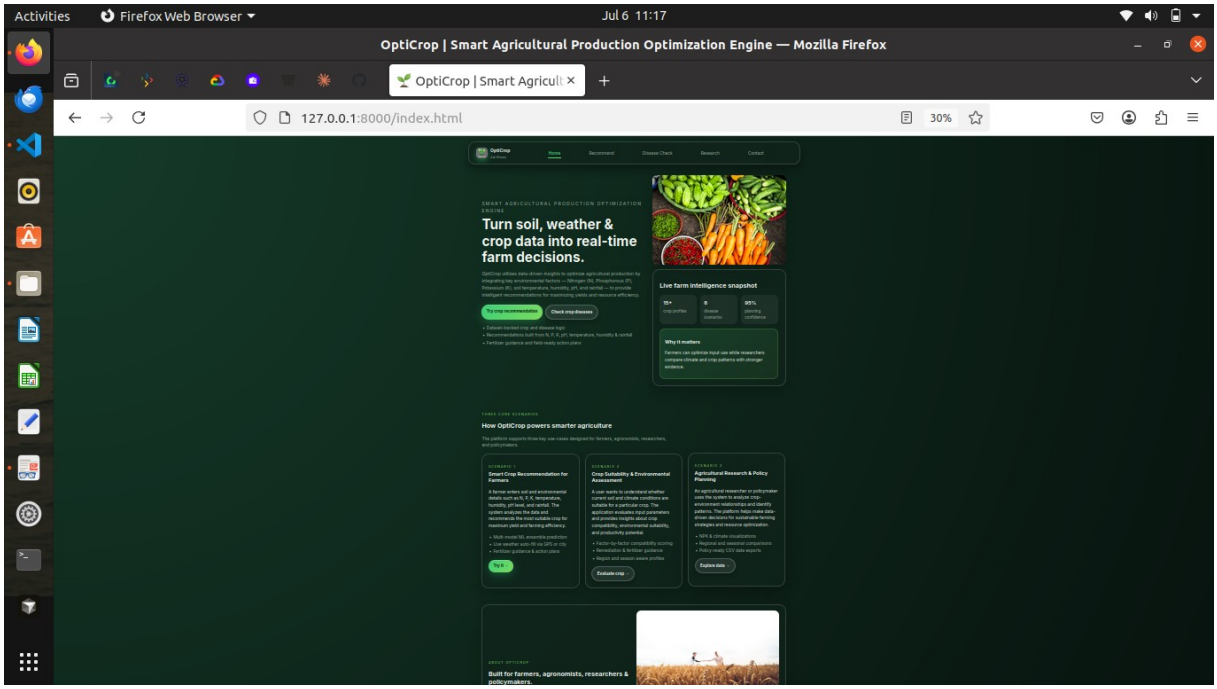

Figure 1: OptiCrop Home Page — index.html

Description: The OptiCrop home page serves as both the landing page and the main navigation hub. It features a dark agricultural-themed design with a hero banner displaying the headline 'Turn soil, weather & crop data into real-time farm decisions.' alongside a live farm intelligence snapshot showing 18+ crop profiles, 8 disease scenarios, and 95% scanning confidence. Below the hero, three scenario cards describe the platform's core use cases: Smart Crop Recommendation for Farmers (Scenario 1), Crop Suitability & Environmental Assessment (Scenario 2), and Agricultural Research & Policy Planning (Scenario 3). A persistent navigation bar links to all five pages. The page is fully responsive with a hamburger menu on mobile screens.

Crop Recommendation & Suitability Page (recommend.html) — http://127.0.0.1:8000/recommend.html

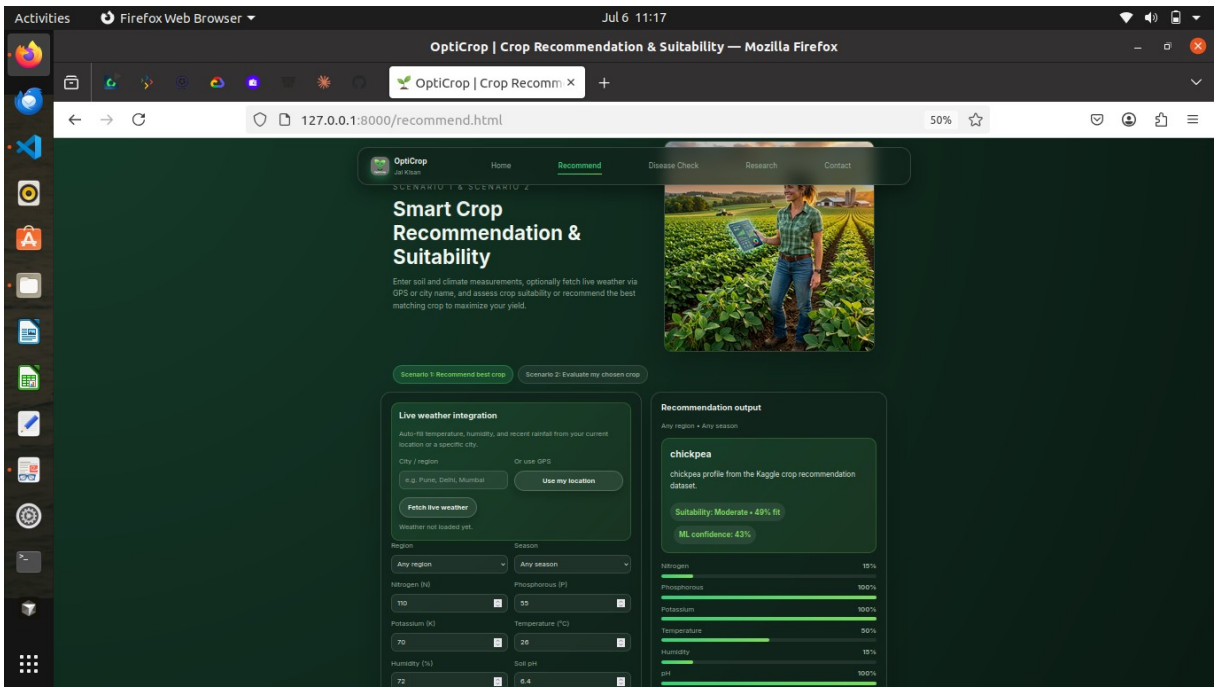

Figure 2: Crop Recommendation & Suitability Page — recommend.html

Description: The Recommend page implements Scenarios 1 and 2 in a two-panel layout. The left panel contains the input form with a Live Weather Integration widget at the top — the farmer can enter a city name or click 'Use my location' (GPS) to auto-fill temperature, humidity, and rainfall via the Open-Meteo API. Below the weather widget, the farmer enters Nitrogen (N), Phosphorous (P), Potassium (K), Temperature (°C), Humidity (%), and Soil pH values, and selects Region and Season from dropdowns. Two mode tabs — 'Scenario 1: Recommend best crop' and 'Scenario 2: Evaluate my chosen crop' — switch between recommendation and suitability assessment modes. The right panel displays the Recommendation Output with the predicted crop name (e.g., Chickpea), suitability label (e.g., Moderate — 49% fit), ML confidence percentage, and per-metric progress bars for Nitrogen, Phosphorous, Potassium, Temperature, Humidity, and pH.

Disease Detection Page (disease.html) — http://127.0.0.1:8000/disease.html

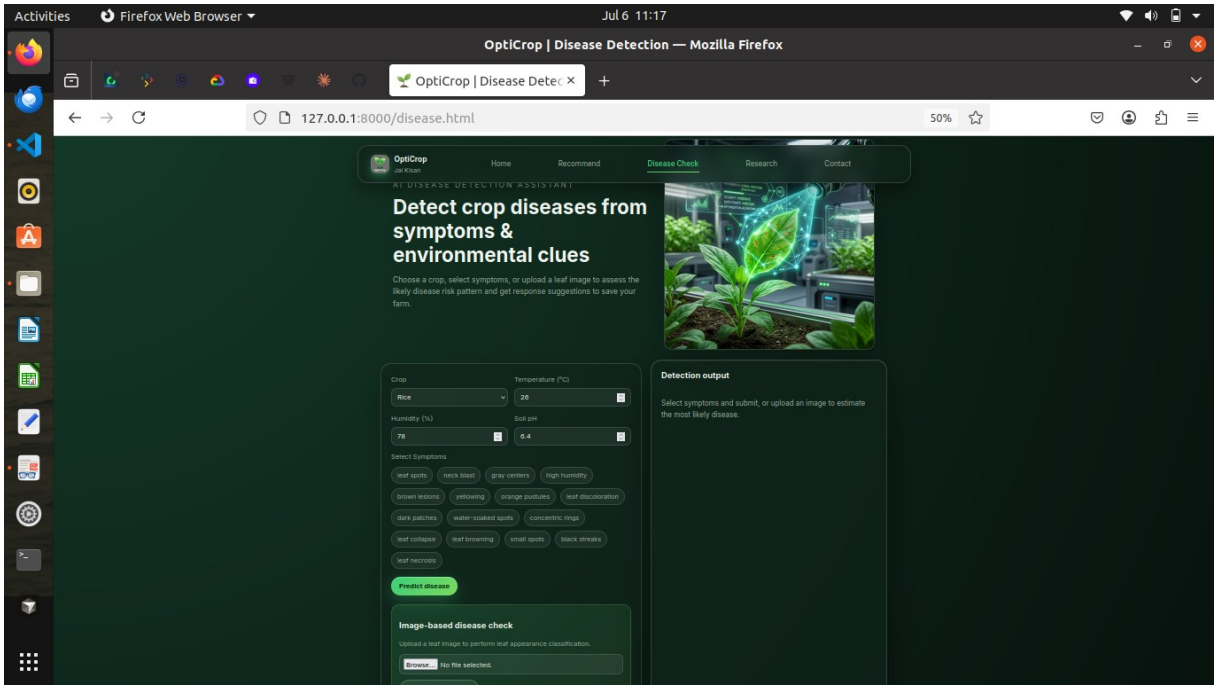

Figure 3: Plant Disease Detection Page — disease.html

Description: The Disease Detection page allows farmers to identify plant diseases through two methods. The upper section contains a symptom-based detection form where the farmer selects their crop (e.g., Rice) from a dropdown, enters temperature (°C) and humidity (%), sets the soil pH, and selects observed symptoms from a tag-style multi-select interface (e.g., leaf spots, neck blast, gray centers, high humidity, brown lesions, yellowing, orange pustules, leaf discoloration, dark patches, water-soaked spots, concentric rings, leaf collapse, leaf browning, small spots, black streaks, leaf necrosis). Clicking 'Predict disease' sends the data to /api/disease and renders the predicted disease name, confidence score, and risk level in the Detection Output panel on the right. The lower section provides an Image-based disease check panel where the farmer can upload a leaf photograph (Browse button) to trigger CNN or pixel-feature RF classification via /api/image-disease.

Research Analytics Dashboard (insights.html) — http://127.0.0.1:8000/insights.html

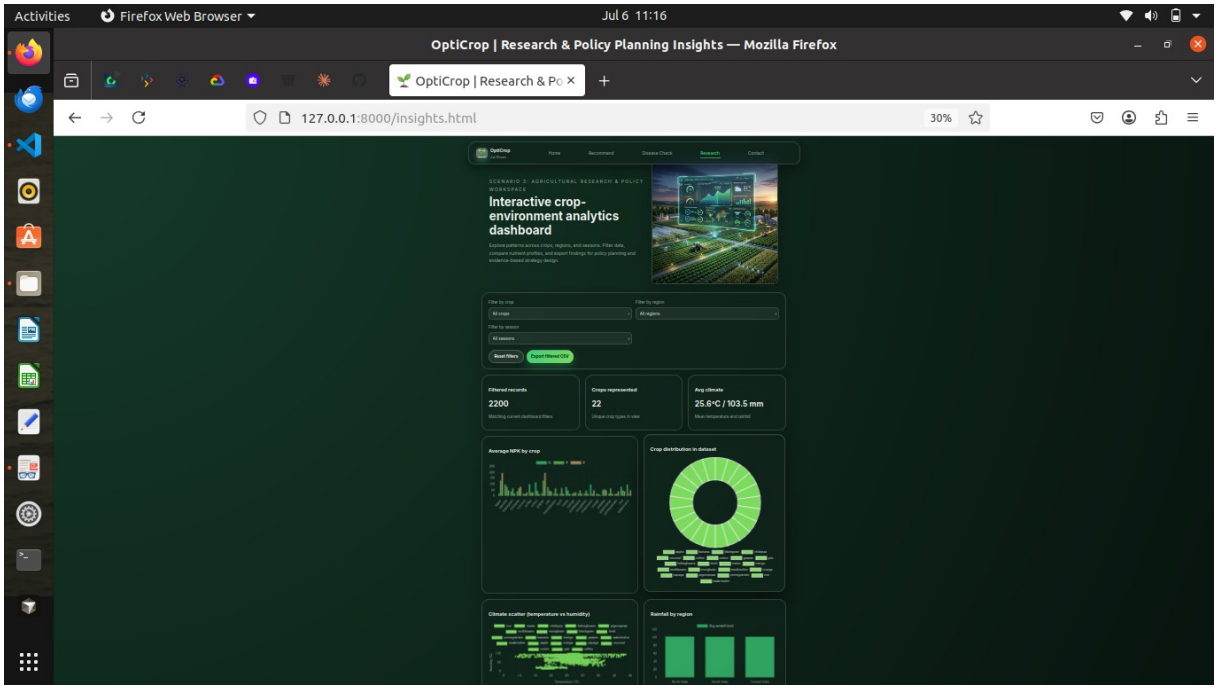

Figure 4: Research & Policy Planning Analytics Dashboard — insights.html

Description: The Research & Policy Planning dashboard implements Scenario 3. It displays four Chart.js visualisations powered by live data from /api/analytics: (1) Average NPK by Crop — a grouped bar chart showing mean Nitrogen, Phosphorous, and Potassium values for all 22 crops; (2) Crop Distribution in Dataset — a donut chart showing the count of each crop in the 2200-row dataset; (3) Climate Scatter (Temperature vs. Humidity) — a scatter plot visualising climate conditions grouped by crop; and (4) Rainfall by Region — a bar chart comparing average 30-day rainfall across North, South, and Central India regions. At the top, three filter dropdowns (Filter by crop, Filter by region, Filter by season) and an 'Export Filtered CSV' button allow researchers to narrow the dataset and download filtered records. A summary row shows the total Filtered Records (2200), Crops Represented (22), and Avg Climate (25.6°C / 103.5 mm) for the current filter selection.

Conclusion:

OptiCrop is a comprehensive ML-powered agricultural optimization platform built with Python's built-in ThreadingHTTPServer and a clean dark-themed five-page web interface that streamlines crop selection, disease detection, and fertilizer management for farmers across India. It uses a Scikit-learn ensemble of Random Forest (n=350), Decision Tree, and SVC models (99.55% accuracy on the test set) for crop recommendation, a separate Random Forest disease classifier for symptom-based plant disease identification, and a pixel-feature Random Forest for image-based leaf disease detection. The real-time weather auto-fill (Open-Meteo API — no key required) and the interactive Chart.js analytics dashboard further distinguish OptiCrop as a practical, farmer-first tool. The project — which included dataset EDA, multi-algorithm comparison, model training, a seven-route Python HTTP backend, and a fully responsive frontend — demonstrates how targeted ML and a carefully structured codebase can directly improve agricultural productivity and farmer livelihoods without dependency on external web frameworks or cloud services.

Looking ahead, OptiCrop offers significant opportunities for expansion. These include integrating multilingual support (Telugu, Hindi, Tamil) to reach a wider farming audience across India; adding IoT soil sensor integration for real-time NPK monitoring without manual data entry; implementing district and season-specific crop recommendation models for hyper-local accuracy; developing a crop yield prediction module (regression model) to estimate harvest quantities before planting; integrating market price data from the Agmarknet / e-NAM API to recommend the most profitable crop based on both yield potential and current market conditions; and building a native Android/iOS offline-capable app for farmers in low-connectivity rural areas. By continuously refining the underlying ML models and enhancing the platform's accessibility, OptiCrop is well-positioned to evolve from a local internship prototype into a comprehensive national agricultural advisory system that empowers millions of Indian farmers to farm smarter, waste less, and earn more.